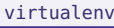


Getting Started with on Linux

 is an extremely fast Python package and project manager written in Rust. It replaces , , , , and  with a single tool.

1 Installation

Standalone installer (recommended)

```
bash
curl -LsSf https://astral.sh/uv/install.sh | sh
```

If `curl` isn't available:

```
bash
wget -qO- https://astral.sh/uv/install.sh | sh
```

```
bash
uv --version
```

2 Create your first project

```
bash
uv init my-project
cd my-project
```

This generates:

```
bash
my-project/
├── .python-version  # pinned Python version
├── pyproject.toml   # project metadata & dependencies
├── README.md
└── main.py
```

Run your code —  creates the virtual environment (`.venv/`) and installs dependencies automatically on first run:

```
bash
uv run main.py
```

3 Managing Python versions

 downloads and manages Python interpreters for you:

```
bash
```

```
uv python list           # available & installed versions
uv python install 3.12    # install a specific version
uv python install 3.11 3.12 3.13 # several at once
uv python find 3.12       # print the path to an installed version
uv python uninstall 3.11  # remove one
```

Interpreters are stored under `~/.local/share/uv/python/`.

Pin a version for the current project

```
bash
```

```
uv python pin 3.12
```

This writes `3.12` into `.python-version`; every `uv run` / `uv sync` in this directory then uses it.

Use a version ad hoc

```
bash
```

```
uv venv --python 3.11      # create a .venv with Python 3.11
uv run --python 3.13 main.py # run one command with another version
uv init my-project --python 3.12 # new project pinned to 3.12
```

4 Adding and removing packages

```
bash
```

```
uv add numpy              # add one package
uv add numpy pandas matplotlib # add several at once
```

Each `uv add` updates `pyproject.toml`, refreshes the `uv.lock` lockfile, and installs into `.venv` — usually in well under a second.

Version constraints

```
bash
```

```
uv add "pandas>=2.2"
uv add "numpy==2.1.0"
uv add "requests~2.32"
```

Extras, dev dependencies and other sources

```
bash
```

```
uv add "pandas[excel]"           # optional extras
uv add --dev pytest ruff          # development-only dependencies
uv add --group docs mkdocs        # a named dependency group
uv add git+https://github.com/psf/requests # from a Git repository
uv add ./local-package            # from a local path
```

Removing and upgrading

```
bash
```

```
uv remove matplotlib
uv lock --upgrade-package pandas # upgrade a single package
uv lock --upgrade               # upgrade everything within constraints
uv tree                         # inspect the dependency tree
```

Sync an existing project

After cloning a repo that already has a `uv.lock`:

```
bash
```

```
uv sync          # install the exact locked dependencies
uv sync --frozen  # fail instead of re-locking (useful in CI)
```

5 Running code and tools

```
bash
```

```
uv run main.py    # run a script in the project env
uv run pytest      # run an installed dev tool
uv run python      # REPL with your dependencies available
```

6 Cheat sheet

Task	Command
Install uv	<code>curl -Lsf https://astral.sh/uv/install.sh sh</code>
New project	<code>uv init myapp</code>
List Python versions	<code>uv python list</code>
Install Python 3.12	<code>uv python install 3.12</code>
Add packages	<code>uv add numpy pandas</code>
Remove a package	<code>uv remove numpy</code>
Install locked deps	<code>uv sync</code>
Run a script	<code>uv run main.py</code>
Update uv	<code>uv self update</code>

Documentation: <https://docs.astral.sh/uv/>